Введение

Эффективное планирование технического обслуживания — ключевой компонент эксплуатации крупномасштабных инфраструктурных систем, таких как телекоммуникационные сети, электростанции, коммунальные объекты, транспортные активы и промышленное оборудование. Хотя большинство объектов инфраструктуры работают автономно, они требуют периодического проведения ТО сервисный персоналом.
Одной из главных проблем при планировании ТО является проектирование зон обслуживания. Неэффективно сформированные зоны могут привести к избыточным затратам времени на перемещения, неравномерному распределению нагрузки и нерациональному использованию ресурсов. При этом практические решения должны учитывать не только географическую близость, но и потребность в ТО во времени, а также географическую целостность территорий обслуживания.
В рамках данного соревнования участникам предлагается разработать алгоритмы кластеризации объектов инфраструктуры на зоны обслуживания, привязав их к сервисным офисам. Цель — минимизировать транспортные расходы, максимизируя число обслуженных объектов, обеспечив сбалансированное распределение нагрузки и географическую связность зон.

Постановка задачи

Дано:

- Множество объектов инфраструктуры, заданных географическими координатами и важностью обслуживания.
- Множество сервисных офисов, для каждого:
  - Географические координаты
  - Стоимость открытия
  - Максимальное число обслуживаемых объектов
- Матрица расстояний (или времени в пути) по дорожной сети между объектами и потенциальными офисами.
- Список работ по ТО, которые необходимо выполнить на каждом объекте. Для каждой работы указаны:
  - объект, на котором она выполняется;
  - требуемый навык рабочего.
- Список специалистов, доступных для назначения с указанием имеющихся навыков

Условия

Каждый объект должен быть закреплен не более чем за одним сервисным офисом. Требуется определить:

- Какие офисы необходимо задействовать.
- Распределение объектов по сервисным офисам.
- Назначения специалистов по офисам.

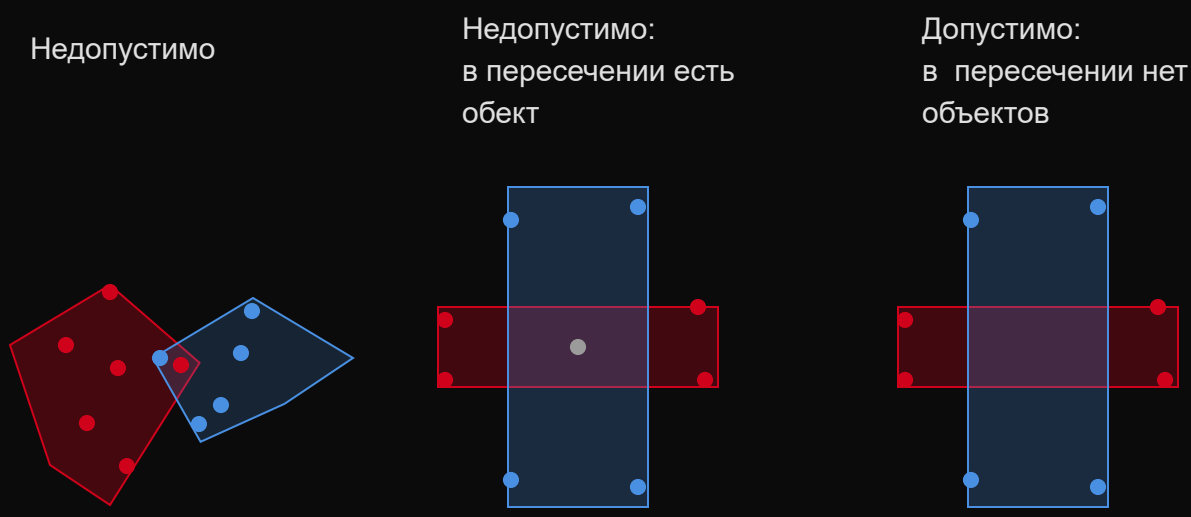
Итоговая стоимость обслуживания рассчитывается на основе предоставленных расстояний (или времени в пути) по дорожной сети, а не евклидовых расстояний. Помимо минимизации расходов на перемещения, сформированные зоны должны обеспечивать сбалансированную нагрузку.

Балансировка объектов
Количество объектов, закрепленных за каждым сервисным офисом, должно быть примерно одинаковым. Это строгое ограничение.

Балансировка нагрузки ТО
Суммарная нагрузка по ТО в каждой зоне должна быть примерно одинаковой. Нагрузка измеряется как число задач, запланированных к выполнению в данном кластере. Это мягкое ограничение.

Геометрические ограничения
Зоны обслуживания должны соответствовать дополнительным географическим требованиям:

Выпуклость: кластеры должны иметь выпуклую географическую форму (кластер включает в себя сам офис).
Отсутствие пересечений: никакой объект не может лежать в пересечении двух зон.
Эти ограничения строгие



Приветствуется разработка точных методов оптимизации, эвристик, метаэвристик, подходов на основе машинного обучения или гибридных решений.

Оценка решений

Решения будут оцениваться по взвешенной комбинации нескольких критериев:

- Транспортные расходы: минимизация суммарных затрат на перемещения по дорожной сети между объектами и закрепленными офисами.
- Расходы на открытие офисов: минимизация суммарной стоимости открытых офисов.
- Выгода от обслуживания: максимизация суммарной важности обслуженных объектов.
- Баланс нагрузки: минимизация дисбаланса нагрузки между зонами.
  При этом решение должно удовлетворять следующим ограничениям:
- Каждый объект назначен не более чем одному офису
- Размер каждого кластера не должен превосходить максимальное значение для данного офиса
- Баланс распределения объектов: количество объектов у любых 2 офисов не должны различаться более чем на A.
- Геометрия и соблюдение ограничений: зоны обслуживания должны быть выпуклыми и не должны пересекаться по объектам.

Итоговая постановка:

$$\sum_{s \in Assinged} reward(s) - \sum_{o,s \in Assignment} distance(o,s) - \sum_{o \in Open} cost(o) -$$
$$-\alpha \cdot (\max_{o \in Open} tasks(o) - \min_{o \in Open} tasks(o)) \to \max$$

$$Assigned(o) \le maxSize(o) \quad \forall o \in Open$$
$$|Assigned(o_1) - Assigned(o_2)| \le A \quad \forall o_1, o_2 \in Open$$
$$isConvex(Zone(o)) \quad \forall o \in Open$$
$$Zone(o_1) \cap Zone(o_2) = \emptyset \quad \forall o_1, o_2 \in Open$$

Победителем признается команда, набравшая наибольшее итоговое количество баллов. В случае равенства баллов (что маловероятно), выбор победителя будет осуществляться комиссией жюри.

Формат данных

На входе для удобства будет дан файл `instance.json` со схемой:

```
{
    "sites": [{
        "x": float,
        "y": float,
        "reward": float,
        "tasks": [int] // Список задач, каждая характеризуется навыком
    }],// Список обеъктов
    "offices": [{
        "x":float,
        "y":float,
        "cost":float,
        "maxSize":int
    }],
    "workers": [[int]],// Список работкиков, каждый работник характеризуется списком номеров навыков>
    "distances": [[float]],// Матрица расстояний офис-объект: строка отвечает номеру офиса, столбец --- номеру объекта
    "maxDifference": int,// A
    "balanceCoefficient": float// alpha
}
```

Формат результата

В результате работы программа должна создать файл `solution.json` со схемой:

```
[{
    "officeId": int, // Номер офиса. Нумерация с 0
    "sites": [int], // Список назначенных обектов
    "workers": [int] // Список назначенных специалистов
 }] // Список открытых офисов
```

Для согласованности результатов вам будет предоставлен проверочный скрипт, выполняющий те же проверки, что и система. Его использование в программе необязательно.

Тестирование

Тестирование производится в системе NSUTS. Решение будет запускаться в Docker контейнере со следующим набором программ:

```
python 3.14
gcc 15
julia 1.13
openjdk 26
go 1.26
rust 1.93

cmake 4.2
gradle 4.4
maven 3.9
cargo 1.93

scip 10.1
highs 1.15
z3 4.13
nlohmann-json 3.12
boost 1.90
eigen 3.4
mlpack 4.7
```

В папке `samples` приведено более подробное руководство по запуску а также примеры.

Для запуска требуется отправить zip архив с `Makefile` с 2 задачами: `build` и `run`. Первая существует для компиляции и скачивания зависимостей, во время её выполнения есть доступ в интернет, но нет доступа к тестовым примерам. Во время

выполнения `run` доступ в интернет отсутствует. Время работы `run` ограниченно 300 секундами.
Пример:

```
build:
    python3 -m venv .venv
    .venv/bin/pip install -q -r requirements.txt

run:
    .venv/bin/python solve.py
```

```
build:
    g++ -O3 -osolve solve.cpp

run:
    ./solve
```

Любая "разведка" сервера, попытки скачать файлы примеров, любая сетевая активность во время работы алгоритма, попытки прочитать иные файлы кроме `instance.json`, записать файлы иные, кроме `solution.json` **строго запрещены** (никаких кэшей между запусками, warmstart и т.д.) и караются дисквалификацией команды.